arXiv:2605.20923v2 [cs.LO] 10 Sep 2026

Causal Past Logic for Runtime Verification of Distributed LLM Agent Workflows

Benedikt Bollig

Université Paris-Saclay, CNRS, ENS Paris-Saclay, LMF, Gif-sur-Yvette, France

Abstract. We study runtime monitoring for distributed LLM-agent workflows. In an asynchronous execution, a decision can only depend on events that are causally visible to the lifeline that makes it: an event that appears earlier in some log may still be unknown locally. We extend the ZipperGen agent-workflow framework with Causal Past Logic (CPL), an adaptation of PT-DTL to guards in if-constructs and while loops. In addition to standard past-time modalities such as previous and since, a guard can inspect the latest causally visible event of another lifeline and selected variables stored there. The owner evaluates the guard online to select the next branch or loop step. We adapt the knowledge-vector monitor to ZipperGen and prove that the locally computed monitor value coincides with the denotational semantics of the guard at the current event.

Keywords: Runtime verification · Message sequence charts · LLM agents · Distributed systems · Causal order

1 Introduction

Several agents based on large language models (LLMs) can cooperate on a task, each with its own tools and local context. Suppose a patch is changed after it has passed the tests. The earlier test result should not justify merging the new version. The committer must therefore check which version the result concerns, using the information it has received so far.

We use runtime verification [5,7,34] to evaluate temporal conditions at workflow decisions. Classical runtime verification often uses temporal specifications and raises an alarm when an execution violates them. Temporal guards can also guide distributed workflow enactment [50]. For example, a committer lacking a clean test result in its causal past can block the merge or ask a human reviewer to decide.

We develop this idea for ZipperGen [12]. ZipperGen programs are written as global workflows and projected to local agent programs that communicate asynchronously. ZipperGen follows the choreographic approach [15, 29, 39, 49], where the programmer writes the coordination from a global point of view, and the framework derives the local behavior of the individual agents. We use the term lifeline for the local participant, usually an agent, that executes its part of the workflow. In ZipperGen, branch and loop decisions have an explicit owner

lifeline. The projection inserts control broadcasts so that the lifelines that need to know a decision are informed about it. It coordinates choices, but remote events may still be unknown to the lifeline making a decision.

We extend ZipperGen guards with Causal Past Logic (CPL), a PT-DTL-style past-time logic [47] interpreted over message sequence charts (MSCs). A guard is written as $\varphi@A$ and is evaluated locally by lifeline A, which owns the corresponding branch or loop. CPL formulas can express causal-temporal facts such as "the causal past contains a test pass that remains valid at the latest visible TestRunner event". They can also combine such facts with identity checks, for instance whether the visible test result concerns the same merge candidate as the current proposal.

Contributions. We adapt PT-DTL [47] to source-level ZipperGen guards. At an owner-side choice event, a guard may inspect the latest causally visible event of another lifeline and selected variables stored there. Its value selects the next branch or loop step. The monitor follows the knowledge-vector scheme of [47], using full vector clocks and latest-value views. A correctness proof for formula-only ptDTL monitoring is given in [45]. We prove that the monitor used here agrees with the denotational semantics of CPL, implement the construction in ZipperGen (`https://zippergen.io`), and measure its runtime and metadata costs.

Mechanization. We mechanized the definitions of MSCs, CPL satisfaction, the local evaluator, latest-value monitor state, the operational event update, and the monitor-correctness theorem in Lean 4 [41]. Lean represents monitor views as extensional functions, using propositions for formula values and local event indices for clock components. Finite-table refinement, executable Boolean evaluation, and the size bounds are outside the mechanization. Badges next to definitions and results open the corresponding source files.

Outline. Section 2 motivates the approach with an example. In Section 3, we define executions in terms of MSCs. Section 4 defines Causal Past Logic. Section 5 presents the online monitoring algorithm and proves correctness. Section 6 describes the implementation and gives a preliminary evaluation, and Section 7 discusses related work. We conclude in Section 8.

2 A Distributed Code Review Example

In an LLM agent workflow, agents exchange messages over dedicated channels and maintain separate local memories. In the formal development, we call each agent a *lifeline*. In this section, we use the two terms interchangeably.

Consider a coding-agent workflow in which a patch is checked by several specialized agents. A TestRunner executes the test suite, a Security agent checks for vulnerabilities, and a Committer decides whether a merge candidate may be merged. These agents run concurrently and report their results asynchronously.

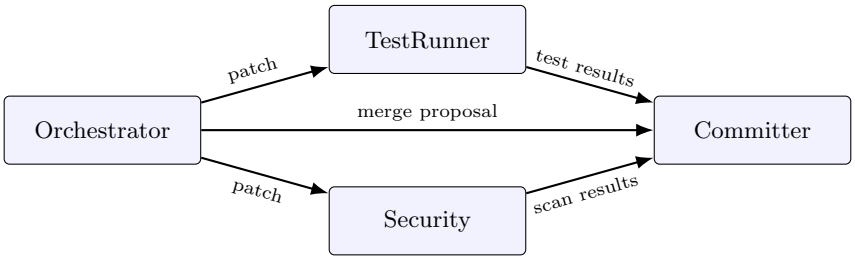


Fig. 1. Architecture for concurrent code review. The Orchestrator distributes the patch to the TestRunner and Security agent, and sends a merge proposal to the Committer when a candidate is ready to be considered. The proposal triggers the Committer's decision procedure. The reviewers run in parallel and may send updated assessments after their first report.

They may also send updated assessments after an initial result. For example, the required pre-merge checks may pass, while a subsequent check detects a regression. Similarly, a security scan may first clear a patch and later report a vulnerable dependency.

The Committer may merge a pull request only if, in its causal past, the required tests have passed with no later visible failure, and the security scan has cleared the patch with no later visible critical issue. Both results must concern the same merge candidate as the current proposal.

Figure 1 shows the architecture. The merge proposal is an ordinary message to the Committer. Its receive event stores the proposed candidate locally and triggers the decision procedure. The guard is evaluated at the following owner-side choice event. It checks the latest visible state of each reviewer, and also checks that these states refer to the current candidate:

$$\begin{aligned} &@\mathsf{TestRunner}.\mathit{candidate} = \mathit{candidate} \ \wedge \ @\mathsf{Security}.\mathit{candidate} = \mathit{candidate} \\ \wedge\ &@\mathsf{TestRunner}((\mathit{status} \neq \mathsf{failed} \wedge \mathit{status} \neq \mathsf{pending}) \ \mathsf{S} \ \mathit{status} = \mathsf{passed}) \\ \wedge\ &@\mathsf{Security}((\mathit{status} \neq \mathsf{critical} \wedge \mathit{status} \neq \mathsf{pending}) \ \mathsf{S} \ \mathit{status} = \mathsf{cleared}). \end{aligned}$$

Here *candidate* is the merge candidate stored at the current Committer event. The term $@A.x$ denotes the value of variable x at the latest event of lifeline A that is causally visible to the Committer. Thus, the two equalities check that the latest visible reviewer episodes concern the current candidate. The formula $@A(\varphi)$ evaluates φ at that same latest visible event of A. The temporal subformulas, where S is the past-time since operator, are evaluated along the local histories of the review lifelines.

Each reviewer processes candidate episodes sequentially. The TestRunner status ranges over pending, checking, passed, and failed. When it starts candidate c, one event atomically sets $\mathit{candidate} := c$ and $\mathit{status} :=$ pending. This pending event delimits the new episode. The value passed records that the required pre-merge checks have cleared, and later checking events may record additional testing. The temporal condition remains true through such events. A later failed

Listing 1. Complete CPL guard and owner-side choice in ZipperGen.

```
not_pending = Here.status != "pending"

def review_ok(A, bad, good):
    return At[A](since(
        (Here.status != bad) & not_pending,
        Here.status == good))

merge_guard = (
    (At[TestRunner].candidate == Here.candidate)
    & (At[Security].candidate == Here.candidate)
    & review_ok(TestRunner, "failed", "passed")
    & review_ok(Security, "critical", "cleared"))

@workflow
def merge_candidate(candidate: str @ Orchestrator) -> str:
    ...
    if merge_guard @ Committer:
        Committer: decision = approve(candidate)
    else:
        Committer: decision = reject(candidate)
```

event invalidates the pass, while a later pending event begins another episode and prevents the earlier pass from witnessing the condition. A new passed event is then required. Together, the candidate equality identifies the current episode and the temporal condition checks its status history.

The Security lifeline analogously uses pending, scanning, cleared, and critical, and resets its candidate and status atomically at the start of each review. Each reviewer lifeline has one current candidate. Sequential candidate episodes share that lifeline, and simultaneous independent reviews use separate reviewer lifelines or candidate-indexed application state.

The helper `review_ok` in Listing 1 constructs the two episode conditions above. The resulting guard directly controls the branch owned by the Committer.

Reviewer reports make the corresponding reviewer states causally visible to the Committer. The guard itself is stated over those reviewer states. A Committer-local encoding would have to reconstruct the reviewers' local orders from received reports, for example using sequence numbers or causal metadata. This is especially relevant when reports are propagated through intermediate participants or delivered out of order: their arrival order at the Committer may differ from the reviewer's local order.

In Figure 2, the required-check result has reached the Committer before the merge proposal. A subsequent check has already found a failure, but the corresponding update remains in transit. Therefore, the latest TestRunner state visible to the Committer is still the required-check pass. At the decision point, the formula $@\mathsf{TestRunner}((\mathit{status} \neq \mathsf{failed} \wedge \mathit{status} \neq \mathsf{pending})\ \mathsf{S}\ \mathit{status} = \mathsf{passed})$

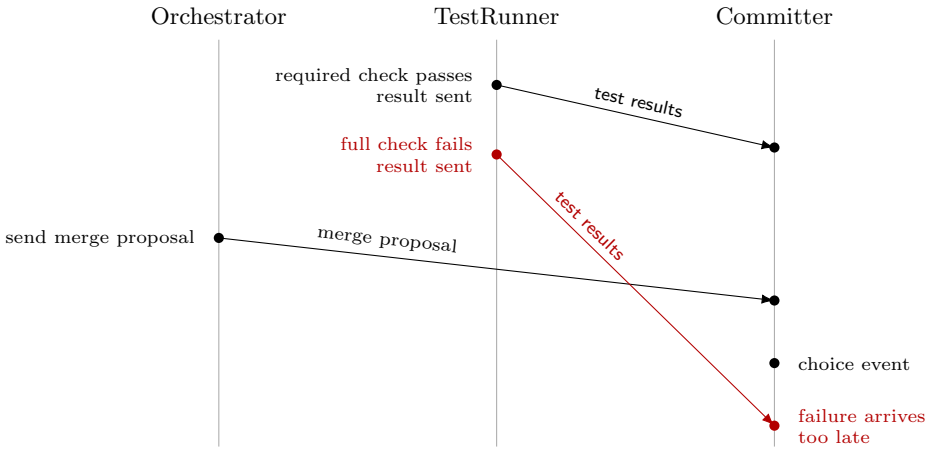


Fig. 2. Causal structure at a merge decision. The Committer sees the test pass at the choice event, while the later failure remains in transit. The earlier assignment messages and local status-update events are omitted. The latest visible state on the omitted Security lifeline satisfies its guard conditions.

holds, and the formula @TestRunner.*candidate* = *candidate* identifies the visible testing episode as belonging to the proposed candidate. The pending reset at the start of that episode keeps the temporal witness within the same episode.

A monitor using a sequential log ordered by send time would see the failure before the decision and block the merge. The Committer evaluates its guard from the information causally visible at the decision point. Here, merging becomes enabled when the required checks pass and remains enabled until a later failure becomes visible.

3 Message Sequence Charts

We model agent-workflow executions as *message sequence charts* (MSCs). An MSC consists of *events* distributed over lifelines. Each event represents one observable step: sending or receiving a message, executing a local action, or making a branching decision. Events on the same lifeline are linearly ordered. Messages are represented by matching send and receive events. Unmatched sends remain in transit in prefixes. The causal order combines local successors with messages. Figure 2 shows a small MSC for the merge scenario: the Orchestrator sends a merge proposal to the Committer, the TestRunner sends a result that arrives before the proposal, and a later update is still in transit at the choice event.

We enrich each event with a *valuation*. The valuation at an event records the local state of the corresponding lifeline after that event. It is a mapping from variables to values, for instance the current merge candidate or the latest test status. The MSC semantics specifies where formulas are evaluated and which

valuations they may inspect. Agent implementations determine their internal store semantics.

Let $\mathscr{L}$ be a finite set of lifelines.

Definition 1 (Message Sequence Chart (MSC) FORMALIZED Lean). *A* Message Sequence Chart (MSC) *over $\mathscr{L}$ is a tuple $M = (E, \to, \lhd, \mathsf{kind}, \mathsf{pid}, \mathsf{val})$, where E is a finite set of events, $\to \subseteq E \times E$ is the local successor relation, $\lhd \subseteq E \times E$ is the message relation, $\mathsf{kind} : E \to \{\mathsf{act}, \mathsf{recv}, \mathsf{choice}\} \uplus (\{\mathsf{send}\} \times \mathscr{L})$ classifies events. A send event has kind (send, B), where B is its intended receiver. The map $\mathsf{pid} : E \to \mathscr{L}$ assigns each event to its lifeline, and val assigns an event valuation to each event. For $A \in \mathscr{L}$, define $E_A = \{e \in E \mid \mathsf{pid}(e) = A\}$ and $\to_A = \to \cap (E_A \times E_A)$. The tuple satisfies the following conditions.*

(i) The relation $\to$ is local: if $e \to f$, then $\mathsf{pid}(e) = \mathsf{pid}(f)$.

(ii) For each A, the relation $\to_A$ is the immediate-successor relation of a finite linear order on E_A. We write $\leq_A$ for its reflexive transitive closure and $<_A$ for the corresponding strict order.

(iii) The message relation $\lhd$ is a partial matching between send and receive events. If $s \lhd r$, then $\mathsf{kind}(s) = (\mathsf{send}, \mathsf{pid}(r))$, $\mathsf{kind}(r) = \mathsf{recv}$, and $\mathsf{pid}(s) \neq \mathsf{pid}(r)$. Every receive has exactly one matching send, and every send has at most one matching receive. Unmatched sends are messages still in transit, but their intended receiver is still given by their event kind. Message delivery may be non-FIFO.

(iv) The graph generated by local successor edges and message edges is acyclic.

Unless stated otherwise, let M be an MSC over $\mathscr{L}$.

The Lean development formalizes event kinds, message matching, and the semantic interface used by CPL: lifelines, local order, causal order, valuations, and the previous-local and latest-visible navigation operations.

The causal order $\leq_M$ is the reflexive transitive closure of local successor edges and message edges. By acyclicity, $\leq_M$ is a partial order. We write $e <_M f$ if $e \leq_M f$ and $e \neq f$.

For an event e, we write $\nu_e := \mathsf{val}(e)$. Thus $\nu_e(x)$ is the value of variable x at event e. One can think of ν_e as the local store of the lifeline after event e. In ZipperGen, actions and receives may update this store, while send and choice events leave it unchanged. For choice events, ν_e is the store on which the guard is evaluated.

4 Causal Past Logic

Definition 2 (CPL syntax FORMALIZED Lean). Variable terms t, atomic conditions α, *and* CPL formulas φ *are given by the following grammar:*

$$t ::= x \mid @A.x \qquad \alpha ::= p(t_1, \ldots, t_n)$$
$$\varphi ::= \alpha \mid @A(\varphi) \mid \mathsf{Y}\varphi \mid \varphi_1 \,\mathsf{S}\, \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \neg\varphi.$$

Here, x ranges over workflow variables and p ranges over application-level predicate symbols. We use standard mathematical notation for common predicates. Thus $x < 10$ abbreviates a unary predicate on x, $\mathit{status} = \mathsf{approved}$ abbreviates a unary predicate on *status*, and $@\mathsf{TestRunner}.\mathit{candidate} = \mathit{candidate}$ abbreviates a binary equality predicate applied to the two displayed terms. Fixing any CPL formula χ, we use the Boolean abbreviations $\mathsf{true} := \chi \vee \neg\chi$ and $\mathsf{false} := \chi \wedge \neg\chi$.

Moreover, Y and S are the usual previous and since operators from past-time linear temporal logic, interpreted along the current lifeline. The operator $@A(\varphi)$ is the causal part: it evaluates φ at the latest visible event of lifeline A. The term $@A.x$ is different. It reads the value of x at that event and can only occur inside atomic conditions.

Let $e \in E$, and let $A = \mathsf{pid}(e)$. Define the previous local event, if it exists, by $\mathsf{last}_{\mathsf{loc}}(e) = \max\{f \in E_A \mid f <_A e\}$. For any lifeline B, define the latest visible event of B from e by $\mathsf{last}_B(e) = \max\{f \in E_B \mid f \leq_M e\}$. Both maxima are unique when they exist, since events on a lifeline are linearly ordered.

For a term t, let $[\![t]\!]_M(e)$ denote its value when evaluated at event e, if defined. Since ν_e records the local store after e and is total, $[\![x]\!]_M(e) = \nu_e(x)$ is always defined. If $\mathsf{last}_A(e)$ exists, then $[\![@A.x]\!]_M(e) = \nu_{\mathsf{last}_A(e)}(x)$. Otherwise, the cross-lifeline term is undefined. If any term in a predicate atom or comparison is undefined, then the atom is false. In particular, if no latest visible A-event exists, then any atomic condition using $@A.x$ is false. Thus $@A.x$ always reads the latest visible value of x on lifeline A. This is the current event itself when e is on A, and the latest causally visible A-event otherwise. Strict movement is expressed separately with the local previous operator Y. For example, $@A(\mathsf{Y}\varphi)$ moves to the latest visible A-event and then one local step back.

The operators $@A(\varphi)$ and $@A.x$ correspond to the remote formulas and remote expressions of PT-DTL [47], adapted to MSC events. PT-DTL assumes that initial remote values are known throughout the system. Here, a remote term with no visible event is undefined and makes its atom false.

The application fixes the interpretation of each predicate symbol p as a relation on data values. The term semantics above supplies its arguments. In particular, an unqualified variable is read from ν_e, while an $@A.x$ term is resolved through the causal structure of the MSC.

Definition 3 (CPL semantics FORMALIZED Lean **).** *The satisfaction relation $M, e \models \varphi$, defined for $e \in E$, is given inductively as follows. For an atomic condition $\alpha = p(t_1, \ldots, t_n)$, we have $M, e \models \alpha$ iff all $v_i = [\![t_i]\!]_M(e)$ are defined and $p(v_1, \ldots, v_n)$ holds. The previous and latest-visible cases are*

$$
\begin{aligned}
&M, e \models \mathsf{Y}\varphi && \text{iff } \mathsf{last}_{\mathsf{loc}}(e) \text{ is defined and } M, \mathsf{last}_{\mathsf{loc}}(e) \models \varphi,\\
&M, e \models @A(\varphi) && \text{iff } \mathsf{last}_A(e) \text{ is defined and } M, \mathsf{last}_A(e) \models \varphi.
\end{aligned}
$$

Boolean connectives are interpreted as usual. If $\mathsf{last}_{\mathsf{loc}}(e)$ is undefined, then $\mathsf{Y}\varphi$ is false. If no latest visible A-event exists, then $@A(\varphi)$ is false. For $e \in E_A$, we have $M, e \models \varphi_1 \,\mathsf{S}\, \varphi_2$ iff there exists $f \in E_A$ with $f \leq_A e$, $M, f \models \varphi_2$, and $M, g \models \varphi_1$ for every $g \in E_A$ with $f <_A g \leq_A e$.

Thus the S navigation is local, while φ_1 and φ_2 may themselves contain causal modalities. For example, $(\mathit{status} \neq \mathsf{failed} \wedge \mathit{status} \neq \mathsf{pending})\ \mathsf{S}\ \mathit{status} = \mathsf{passed}$ holds at a TestRunner event e if the current candidate episode contains a pass followed only by checking or passed events, up to e.

Causal past is a derived notation and is non-strict: $\mathsf{P}\varphi := \bigvee_{B \in \mathscr{L}} @B(\mathsf{true}\mathsf{S}\varphi)$. The lifeline-specific form is $\mathsf{P}_A(\varphi) := @A(\mathsf{true}\ \mathsf{S}\ \varphi)$.

Thus, $\mathsf{P}_A(\varphi)$ holds at e exactly when some event on lifeline A visible from e satisfies φ, and $\mathsf{P}\varphi$ is the disjunction over all lifelines. For instance, the formula $\mathsf{P}_{\mathsf{TestRunner}}(\mathit{status} = \mathsf{passed})$ holds at an event e if some event on the TestRunner lifeline visible from e satisfies $\mathit{status} = \mathsf{passed}$.

The abbreviation $\mathsf{Seen}_A := @A(\mathsf{true})$ tests whether some event of A is visible. If the current event itself is on A, this is true by non-strictness.

5 Online Monitoring

At runtime, each guard is evaluated by its owner lifeline. The monitor must therefore track which events are causally visible at the current event. Each lifeline A maintains a vector clock [22,33,36], which is of the form $\mathsf{vc}_A : \mathscr{L} \to \mathbb{N}$. We use vector clocks after the current event has been processed. Thus, after processing an event $e \in E_A$, the value $\mathsf{vc}_A(B)$ records how many events of B are in the causal past of e, including e itself when $B = A$. This matches the non-strict semantics of $@B(\varphi)$ and $@B.x$: both refer to the latest visible B-event. We let e_k^B denote the k-th event on lifeline B (where indexing starts at 1).

Let Φ be a finite set of CPL formulas, $\mathsf{sub}(\Phi)$ the set of all their subformulas, and $\mathsf{vars}(\Phi)$ the set of variable names that appear in *cross-lifeline* terms $@B.x$ in Φ. For the workflow under consideration, Φ contains all CPL guards that can be evaluated at runtime. All lifelines use the same Φ. Derived shorthands such as P_A are unfolded to primitive CPL syntax before taking subformulas. The monitor state is designed to answer three kinds of local queries: strict previous-local formulas, latest visible formulas on each lifeline, and latest visible field values used by terms $@B.x$. The state follows the knowledge-vector scheme of [47]. We use full vector clocks, which also serve as event indices and presence tests in the correctness proof. This is formalized in the following definition:

Definition 4 (Monitor state FORMALIZED Lean**).** *At each lifeline A, the monitor maintains:*

- *a* vector clock $\mathsf{vc}_A : \mathscr{L} \to \mathbb{N}$.
- *a partial* Boolean latest-value view $\mathsf{view}_A : \mathscr{L} \times \mathsf{sub}(\Phi) \rightharpoonup \{0,1\}$.
- *a partial* variable view $\mathsf{var}_A : \mathscr{L} \times \mathsf{vars}(\Phi) \rightharpoonup \mathsf{Val}$, *where* Val *is the domain of stored values.*
- *a* local store σ_A *for the current event valuation.*

It also uses a temporary previous-local copy *during an event update:* $\mathsf{old}_A(\psi) \in \{0,1\}$ *for each* $\psi \in \mathsf{sub}(\Phi)$. *In a stable state, after an event update has completed, the clock is the presence test for the two latest-value tables: entries with first*

component B are defined exactly when $\mathsf{vc}_A(B) > 0$. If $\mathsf{vc}_A(B) = k > 0$, those entries refer to the k-th event of B. If $\mathsf{vc}_A(B) = 0$, they are absent.

The temporary copy old_A is total. At the first event of A, it is set to 0 for every $\psi \in \mathsf{sub}(\Phi)$.

For a fixed workflow, the Boolean part of the monitor state has size $O(|\mathscr{L}| \cdot |\mathsf{sub}(\Phi)|)$ per lifeline, including the previous-local copy old_A. Variable views add $O(|\mathscr{L}| \cdot |\mathsf{vars}(\Phi)|)$ selected data values. The vector-clock components are unbounded counters. In PT-DTL monitoring, the freshness vector contains entries only for processes named by monitored remote expressions and formulas [47]. The same optimization applies here. The unbounded counters agree with the impossibility of deterministic finite-state gossip in unbounded message-passing systems [10]. In practice, they are bounded by the lifetime of the run or a timeout.

We reason about a run by fixing an arbitrary linear extension of the causal order of the MSC. Each event is processed by the monitor of its own lifeline when that event occurs. The prefix processed before an event is therefore causally closed: if an event has been processed, all of its causal predecessors have been processed as well. Initially, all clock components are zero and the latest-value tables are empty. We assume that applying effect at an event e produces the MSC valuation $\mathsf{val}(e)$, and that a receive obtains the monitor snapshot emitted at its matching send. On each lifeline, the state produced at one event is used at the next. The first event starts from the initial monitor state.

When lifeline A creates a new event, the monitor runs Algorithm 1. For a receive, it first merges the vector clock, Boolean views, and variable views carried by the incoming message. Entries are copied from the message exactly when the message is ahead on a lifeline B. In that case the message clock is positive on B, so the corresponding message entries are defined. The monitor then increments its local clock component, updates the local store, evaluates all subformulas (Algorithm 2), and records the results. All messages carry vector clocks and latest-value views. The algorithm treats user messages and projection-internal control messages uniformly.

The function effect accounts for the event's local store update. In the current ZipperGen runtime, actions and receives may update the store, while sends and choice events use the identity update.

Algorithm 2 computes truth values by structural recursion, following the standard recurrence scheme for past-time operators [27]. During an event update, Eval_A reads the current monitor state of A, including the previous-local snapshot old_A. If there is no previous local event, $\mathsf{Y}\theta$ is false. If no latest visible B-event exists, $@B(\theta)$ is false. The $@B(\theta)$ case is non-strict: for $B = A$ it evaluates θ at the current event, and for $B \neq A$, it first checks whether a B-event is visible and only then reads $\mathsf{view}_A(B, \theta)$. A previous *local* event exists when $\mathsf{vc}_A(A) > 1$. We write $(\sigma_A, \mathsf{var}_A) \models \alpha$ for runtime atomic evaluation: unqualified variables are read from σ_A, terms $@B.x$ are read from $\mathsf{var}_A(B, x)$, and the resulting values are passed to the same application predicate as in the MSC semantics. A missing required field makes the atom false. For exam-

Algorithm 1. Event update and view propagation.

```
procedure ON_EVENT(A, e)
  if kind(e) = recv then
    μ ← message(e)
    for B ∈ 𝒮 do
      if μ.vc(B) > vc_A(B) then
        for ψ ∈ sub(Φ) do
          view_A(B, ψ) ← μ.view(B, ψ)
        end
        for x ∈ vars(Φ) do
          var_A(B, x) ← μ.var(B, x)
        end
      end
    end
    vc_A ← max(vc_A, μ.vc)
  end

  for ψ ∈ sub(Φ) do
    old_A(ψ) ← view_A(A, ψ) if defined, otherwise 0
  end

  vc_A(A) ← vc_A(A) + 1
  σ_A ← effect(e, σ_A)
  for x ∈ vars(Φ) do
    var_A(A, x) ← σ_A(x)
  end

  for ψ ∈ sub(Φ) do
    res_A(ψ) ← Eval_A(ψ, e)
  end
  for ψ ∈ sub(Φ) do
    view_A(A, ψ) ← res_A(ψ)
  end

  if kind(e) = (send, B) then
    emit to B the message (payload(e), vc_A, view_A, var_A)
  end
end
```

ple, the Committer evaluates @TestRunner.*candidate* = *candidate* by comparing $\mathsf{var}_{\mathsf{Committer}}(\mathsf{TestRunner}, \mathit{candidate})$ with its current local candidate.

Suppose $A \neq B$, and let b be the latest B-event visible at the current event of A. In $@B(@A(\varphi))$, the inner formula refers to the latest A-event visible from b, if any. The monitor therefore reads $\mathsf{view}_A(B, @A(\varphi))$, rather than evaluating φ at the current A-event. For $\mathsf{Y}@B(\varphi)$, it reads $\mathsf{old}_A(@B(\varphi))$, the value recorded

Algorithm 2. Formula evaluation at the current event on lifeline A.

```
1  procedure Eval_A(ψ, e)
2    case ψ of
3      α: return (σ_A, var_A) ⊨ α
4      Yθ:
5        return (vc_A(A) > 1) ∧ old_A(θ)
6      @B(θ):
7        if B = A then return Eval_A(θ, e)
8        else if vc_A(B) = 0 then return 0
9        else return view_A(B, θ)
10     θ1 S θ2:
11       prev ← (vc_A(A) > 1) ∧ old_A(θ1 S θ2)
12       return Eval_A(θ2, e) ∨ (Eval_A(θ1, e) ∧ prev)
13     ¬θ: return ¬Eval_A(θ, e)
14     θ1 ∧ θ2: return Eval_A(θ1, e) ∧ Eval_A(θ2, e)
15     θ1 ∨ θ2: return Eval_A(θ1, e) ∨ Eval_A(θ2, e)
16   end
17 end
```

at the previous local event. Newly received information must not be used to recompute that value.

The Lean evaluator uses `none` for an absent view and initializes the previous-local table to false. For states produced by Algorithm 1, these conventions are equivalent to the explicit clock tests in Algorithm 2.

Definition 5 (Coherent pre-evaluation state FORMALIZED Lean**).** *Let $e \in E_A$. The monitor state of A is* coherent before evaluating formulas at e *if the following conditions hold.*

(i) For every lifeline B,

$$\mathsf{vc}_A(B) = |\{f \in E_B \mid f \leq_M e\}|.$$

Thus, if $\mathsf{vc}_A(B) = k > 0$, then the latest visible event of B from e is e^B_k.

(ii) For every $B \neq A$ with $k = \mathsf{vc}_A(B) > 0$, the entries for B are defined and describe e^B_k:

$$\mathsf{view}_A(B, \psi) = 1 \quad \textit{iff} \quad M, e^B_k \models \psi$$

for all $\psi \in \mathsf{sub}(\Phi)$, and

$$\mathsf{var}_A(B, x) = \mathsf{val}(e^B_k)(x)$$

for all $x \in \mathsf{vars}(\Phi)$. If $\mathsf{vc}_A(B) = 0$, then no B-event is visible and all entries $\mathsf{view}_A(B, \psi)$ and $\mathsf{var}_A(B, x)$ are absent.

(iii) For the owner lifeline A, the local store induces the current event valuation: for every variable x,

$$\sigma_A(x) = \mathsf{val}(e)(x).$$

Moreover, $\mathsf{var}_A(A,x)$ *is defined and equal to* $\mathsf{val}(e)(x)$ *for every* $x \in \mathsf{vars}(\Phi)$. *No condition is imposed on the owner Boolean entries* $\mathsf{view}_A(A,\psi)$ *before formula evaluation. They are overwritten by Algorithm 1 after the subformulas have been evaluated.*

(iv) *For every* $\psi \in \mathsf{sub}(\Phi)$, $\mathsf{old}_A(\psi)$ *is the truth value of* ψ *at the previous local event of* A, *if such an event exists, and is false otherwise.*

Lemma 1 (Local evaluation VERIFIED Lean **).** *Let* $e \in E_A$. *Suppose the monitor state of* A *is coherent before evaluating formulas at* e. *Then, for every* $\psi \in \mathsf{sub}(\Phi)$,

$$\mathsf{Eval}_A(\psi, e) = 1 \quad \textit{iff} \quad M, e \models \psi.$$

Proof. The proof is by structural induction on ψ.

Atomic condition α. The runtime evaluates α against the current store σ_A and the latest-visible variable views $\mathsf{var}_A(B,x)$. Unqualified variables are read from σ_A, which agrees with $\mathsf{val}(e)$ by coherence. For $@A.x$, coherence gives the value of x at $\mathsf{last}_A(e) = e$. For $@B.x$ with $B \neq A$, coherence gives the value of x at $e^B_{\mathsf{vc}_A(B)} = \mathsf{last}_B(e)$ when a visible B-event exists. If none exists, both the runtime and the semantics make the atom false. Thus every term has the same definedness and value in both evaluations. Both apply the same interpretation of the predicate symbol, so their truth values agree.

Boolean connectives. The cases for $\neg\theta$, $\theta_1 \wedge \theta_2$, and $\theta_1 \vee \theta_2$ follow directly from the induction hypothesis.

Previous: $\mathsf{Y}\theta$. If $\mathsf{vc}_A(A) = 1$, then e is the first local event of A, so there is no previous local event and both sides are false. Otherwise, coherence says that $\mathsf{old}_A(\theta)$ is the truth value of θ at the previous local event, which is exactly the semantics of Y.

At-operator: $@B(\theta)$. If $B = A$, non-strictness gives $\mathsf{last}_A(e) = e$, and the algorithm recursively evaluates θ at the current event. The result is correct by induction. If $B \neq A$, the algorithm reads the latest-value view $\mathsf{view}_A(B,\theta)$, which is correct by coherence. If no B-event is visible, both sides are false.

Since: $\theta_1 \,\mathsf{S}\, \theta_2$. The algorithm uses the standard recurrence for past-time since over the local order of A, i.e., $\theta_1 \,\mathsf{S}\, \theta_2 \equiv \theta_2 \vee \big(\theta_1 \wedge \mathsf{Y}(\theta_1 \,\mathsf{S}\, \theta_2)\big)$. The current values of θ_1 and θ_2 are correct by the induction hypothesis, and the previous value of the since formula is provided by old_A. □

Lemma 2 (Monitor-state invariants VERIFIED Lean **).** *After Algorithm 1 has processed an event* $e \in E_A$, *the following hold for every lifeline* B:

(i) $\mathsf{vc}_A(B) = |\{f \in E_B \mid f \leq_M e\}|$.

(ii) *If* $\mathsf{vc}_A(B) = 0$, *then all entries* $\mathsf{var}_A(B,x)$ *and* $\mathsf{view}_A(B,\psi)$ *are absent.*

(iii) *If* $\mathsf{vc}_A(B) = k > 0$, *then* $\mathsf{var}_A(B,x)$ *is defined and equal to* $\mathsf{val}(e^B_k)(x)$, *for every* $x \in \mathsf{vars}(\Phi)$.

(iv) *If* $\mathsf{vc}_A(B) = k > 0$, *then* $\mathsf{view}_A(B,\psi)$ *is defined and*

$$\mathsf{view}_A(B,\psi) = 1 \quad \textit{iff} \quad M, e^B_k \models \psi$$

for every $\psi \in \mathsf{sub}(\Phi)$.

Proof. Fix a linear extension of the causal order. We proceed by induction over this order. Equivalently, the induction hypothesis is available for the causally closed prefix processed before the current event.

Base case. Before any event is processed, $\mathsf{vc}_A(B) = 0$ for all B. No event is yet causally visible, so (i) holds. The latest-value tables are empty, so (ii) holds. Conditions (iii) and (iv) are irrelevant since no clock component is positive.

Non-receive event e on A. This covers act, send, and choice events. The algorithm increments $\mathsf{vc}_A(A)$ by one and leaves all other clock components unchanged.

(i) This matches the causal past: since e is not a receive, the only newly visible event is e itself, on lifeline A.
(ii) No entry becomes absent when processing an event. For lifelines whose clock remains zero, the tables remain empty.
(iii) The algorithm writes $\mathsf{var}_A(A, x) \leftarrow \mathsf{val}(e)(x)$ from the current local store, which equals $\mathsf{val}(e^A_{\mathsf{vc}_A(A)})(x)$ since $e^A_{\mathsf{vc}_A(A)} = e$. For $B \neq A$, $\mathsf{var}_A(B, x)$ is unchanged and no new B-event became visible, so the stored value still refers to the correct event by induction.
(iv) Algorithm 1 overwrites $\mathsf{view}_A(A, \psi)$ with the value of ψ at e. The monitor state is coherent before formula evaluation: (i) gives the clock condition, remote views are unchanged and correct by induction, the local store has just been updated to induce $\mathsf{val}(e)$, local variable views have just been updated to $\mathsf{val}(e)$, and old_A contains the previous local truth values. Hence Lemma 1 gives the correctness of the new local Boolean entries. For $B \neq A$, $\mathsf{view}_A(B, \psi)$ is unchanged.

Receive event e on A, matching send s on B. The message carries vc^s_B, view^s_B, var^s_B, the monitor state of B immediately after s. By induction, all four invariants hold for B at s. Since $s <_M e$, every event causally visible to B at s is also causally visible to A at e. Write vc^-_A, view^-_A, and var^-_A for the monitor state on A just before the receive merge.

The algorithm sets $\mathsf{vc}_A(C) \leftarrow \max(\mathsf{vc}_A(C), \mathsf{vc}^s_B(C))$ for each C, then increments $\mathsf{vc}_A(A)$.

(i) For $C \neq A$: the only new non-local causal information at the receive event comes through the matching message edge $s \lhd e$. Hence a C-event visible at e is either already visible to A before the receive, or visible to B at the send event s. By induction, the two counts are $\mathsf{vc}^-_A(C)$ and $\mathsf{vc}^s_B(C)$. Both sets of C-events are prefixes of the local order on C. Therefore their union is the longer prefix, whose size is the maximum of the two sizes.
For $C = A$: any A-event visible to B at s is already a strict local predecessor of e. Indeed, if $e \leq_A a$, then $a \leq_M s <_M e \leq_M a$, a causal cycle. Hence $\mathsf{vc}^s_B(A) \leq \mathsf{vc}^-_A(A)$. The merge therefore does not change the A-component, and the final increment accounts for e itself.
(ii) If the resulting clock on C is zero, then both incoming snapshots have clock zero on C, so both have absent entries by induction and the result remains absent.

(iii) The algorithm copies $\mathsf{var}^s_B(C,x)$ into $\mathsf{var}_A(C,x)$ exactly when $\mathsf{vc}^s_B(C) > \mathsf{vc}^-_A(C)$, i.e., when the sender is strictly ahead on C. By induction, we have $\mathsf{var}^s_B(C,x) = \mathsf{val}(e^C_{\mathsf{vc}^s_B(C)})(x)$, so the copy is correct. When $\mathsf{vc}^s_B(C) \le \mathsf{vc}^-_A(C)$, the local view is already at least as recent as the sender's view, so no copy is needed. The local entry $\mathsf{var}_A(A,x)$ is set to $\mathsf{val}(e)(x)$ as in the non-receive case.

(iv) The same merge applies to $\mathsf{view}_A(C,\psi)$: when $\mathsf{vc}^s_B(C) > \mathsf{vc}^-_A(C)$, $\mathsf{view}^s_B(C,\psi)$ is copied. By induction, we have $\mathsf{view}^s_B(C,\psi) = 1$ iff $M, e^C_{\mathsf{vc}^s_B(C)} \models \psi$. For $C \neq A$, after the vector-clock merge, the stored table entry refers to the correct latest visible event of C. The inequality $\mathsf{vc}^s_B(A) \le \mathsf{vc}^-_A(A)$ established in (i) also ensures that the merge leaves the owner entries $\mathsf{view}_A(A,\psi)$ unchanged. Hence the subsequent assignment to old_A records the truth values at the previous local event. After the merge and the local updates of Algorithm 1, the monitor state is coherent, so Lemma 1 applies as in the non-receive case.

This concludes the proof of Lemma 2. □

Suppose two messages from the same sender reach a receiver in reverse send order. Every event visible at the earlier send is also visible at the later send. By Lemma 2(i), the later snapshot therefore has at least the same value in every clock component. Once it has been merged, the older snapshot cannot replace any remote row. The older message still causes a local receive event: the monitor increments its own clock, applies the local store update, and evaluates the formulas.

For an event $e \in E_A$, let $\mathsf{res}^e_A(\varphi)$ denote the value computed for φ by Algorithm 1 during the update for e. The theorem applies to individual executions. Agreement on branch outcomes at the workflow level follows from ZipperGen's projection discipline.

Theorem 1 (Monitor correctness VERIFIED Lean **).** *Let M be an MSC, fix any linear extension of its causal order, and run the distributed monitors along this order. If $e \in E_A$ is an event on lifeline A, then immediately after the monitor on A has processed e, every formula $\varphi \in \Phi$ satisfies*

$$\mathsf{res}^e_A(\varphi) = 1 \quad \textit{iff} \quad M, e \models \varphi.$$

Proof. After processing e, the algorithm has copied $\mathsf{res}_A(\varphi)$ into $\mathsf{view}_A(A,\varphi)$. By Lemma 2(i), $\mathsf{vc}_A(A)$ is the local index of e, so $e^A_{\mathsf{vc}_A(A)} = e$. Applying Lemma 2(iv) with $B = A$, we get $\mathsf{view}_A(A,\varphi) = 1$ iff $M, e \models \varphi$. Since $\mathsf{res}^e_A(\varphi) = \mathsf{view}_A(A,\varphi)$ immediately after the update, the claim follows. □

6 Implementation and Preliminary Evaluation

Implementation. CPL guards are implemented in a ZipperGen prototype[1], an open-source Python framework for multi-agent LLM coordination. Guards are

[1] https://zippergen.io

Python expressions using `At[B].x` for variable terms, `Here.x` for the store at the current evaluation point, `since` for the past-time condition, and standard Boolean connectives. The prototype keeps the monitor state locally, sends vector clocks and latest-value views with each message, and evaluates guards at owner-side choice events, before executing the selected continuation. It retains the fields that occur in cross-lifeline terms and evaluates each registered formula object once per event in bottom-up order. For the benchmark formulas, the registered object counts coincide with the set-based subformula counts. Listing 1 shows the complete code-review guard and its use in a ZipperGen branch.

Monitor microbenchmark. We measured the time and message data used by the ZipperGen monitor.[2] The first configuration uses the four lifelines and the complete merge guard from Section 2: 16 subformulas and $\mathsf{vars}(\Phi) = \{candidate\}$. The status tests use formula views. Only *candidate* requires a variable view. For each local status test, the benchmark reads `status` directly from the local store instead of using the `Here.status` comparison shown in Listing 1. Because the local store is total, the two forms give the same result, but they need not take the same time.

For the scaling cases, with $1 \le k \le n$, a monitor on $L_{\ell-1}$ stores view entries for $L_0, \ldots, L_{\ell-1}$ and evaluates $\Psi_{n,k} = \bigwedge_{i=0}^{n-1} \big(@L_0.f_{i \bmod k} = i\big)$.

The conjunction follows the structure of the merge guard, whose checks must all hold. Using disjunction would give essentially the same timings because the monitor visits every subformula. The field names repeat cyclically, while the constants distinguish the atoms. Hence $\Psi_{n,k}$ has n atoms, $2n-1$ subformulas, and k variable names. The case $k = 128$ uses $n = 128$.

The three operations are timed separately. Before each measurement, the monitor contains a stored value for every formula and tracked variable on every lifeline. The local case processes an action event. The send case processes a send event and copies the vector clock and the two views that the runtime would attach to an outgoing message. The receive case first advances the synthetic incoming clock entries for $L_0, \ldots, L_{\ell-2}$, then passes the metadata to $L_{\ell-1}$. Its views contain the corresponding rows, so the monitor replaces every remote row. The receive time includes these clock updates. The timings cover only the monitor updates described above. Encoding and transmitting messages, storing runtime state, and processing application payloads are not timed.

For each operation, we discard 50 warm-up events and report the median of the mean event times in nine batches of 300. Garbage collection is disabled during measurement. Message size is the UTF-8 length of the JSON-encoded vector clock and views, with one KB equal to 1,000 bytes. The experiments used CPython 3.12 on an Intel Core i9-9880H running macOS. The timings were stable: the interquartile range for every recorded row was below 2.7% of its median.

The code-review guard takes about 0.05–0.07 ms per event and carries 1.2 KB. Receive time rises from 0.10 to 0.76 ms as n grows from 4 to 64. With

[2] Artifacts and exact ZipperGen revisions: public repository.

Table 1. Time per monitor event and monitor data per message.

Workload	ℓ	Atoms n	Variables k	Local (ms)	Send (ms)	Receive (ms)	Message data (KB)
Code review	4	–	1	0.047	0.069	0.072	1.2
Scaling $\Psi_{n,k}$	2	16	4	0.135	0.158	0.154	1.1
Scaling $\Psi_{n,k}$	32	16	4	0.209	0.506	0.557	17.0
Scaling $\Psi_{n,k}$	8	4	4	0.057	0.095	0.102	1.9
Scaling $\Psi_{n,k}$	8	64	4	0.521	0.754	0.764	14.1
Scaling $\Psi_{n,k}$	8	128	1	0.998	1.430	1.448	27.3
Scaling $\Psi_{n,k}$	8	128	128	1.976	2.979	2.903	58.0

$n = 128$, increasing k from 1 to 128 roughly doubles receive time and message data. Increasing ℓ from 2 to 32 raises message size from 1.1 to 17.0 KB.

Workflow generation. We asked Codex CLI 0.153.4 with `gpt-6-astra`, at low reasoning effort, to generate two workflows from short prose specifications, with three runs per task. Each run started from a fresh project created with `zippergen init`. The agent then read the standard skill, ZipperGen's guide for coding agents. The guide shows a remote-field example but no temporal operator. We gave no further explanation of CPL and no reference guard. In the first task, L_3 reported a coin outcome stored at L_0, while the application payloads contained only an opaque token along $L_0 \to L_1 \to L_2 \to L_3$. All three runs generated `(At[L0].outcome == "heads") @ L3`, passed validation, and returned the expected result for both outcomes.

In the second task, L_0 recorded three status updates before the same chain. A pass remained valid through `checking`, a failure cancelled it, and a later pass restored it. The agent had to recognize this historical condition and place `since` under the remote operator. In runs 1 and 2, it listed the names exported by `zippergen` and read the `since` docstring using `inspect.getdoc`. The third run first wrote `Since`. Validation rejected that name and suggested `since`. The same run then placed `S` outside $@L_0((\cdot))$, thereby evaluating it along L_3's history. The agent's generated tests exposed this second error, and the agent corrected it. All three final workflows used $@L_0((\mathit{status} \neq \mathsf{failed}\ S\ \mathit{status} = \mathsf{passed}))$, passed validation, and returned the expected result for all 27 three-update histories. Together, the generated workflows illustrate remote-field access and the evaluation of a local S-formula at a remote lifeline. Episode identity is not tested.

7 Related Work

Runtime verification is usually defined over sequential traces [5,7,34]. Java-MOP, AspectJ-based temporal assertions, and Control-Flow Temporal Logic connect specifications more closely with source code and control flow [16,20,51]. Runtime enforcement may prevent violations [46].

Recent runtime checks for LLM-agent systems inspect action or input-output histories [2,32,42,52,53]. CPL guards instead depend on the causal history visible to the agent making the decision.

PT-DTL [47] is the closest predecessor. Its remote primitive, inspired in part by the message-passing logic of Meenakshi and Ramanujam [38], lets a process refer to formulas and expressions at the latest causally known state of another process. Its knowledge-vector monitor carries the required values with application messages. Scheffel and Schmitz later extend this approach with three-valued monitoring and give a correctness proof for a formula-only variant of PT-DTL [45]. CPL adopts this approach for valued MSCs and follows Sen et al. in carrying both formula and selected variable values. Scheffel and Schmitz use the first state for an unseen process, while CPL leaves its view absent. Lemma 2 and its Lean proof cover this setting, with clock entries represented by local event indices.

Singh also places temporal guards on events in multiagent workflows [50], while DistAlgo incrementally maintains conditions over sent and received messages [35]. CPL also uses temporal information to guide control flow, but its guards follow PT-DTL's latest-visible formula and value semantics.

ZipperGen [12] draws on choreographic programming and multiparty session types [15, 29, 30, 39, 49]. This setting also includes projected assertions and runtime monitoring [8,9,14,17,28]. For example, Scribble/Python monitors endpoint conversations and allows them to be interrupted [21], while Pact studies choices in agent workflows [25]. In reversible and adaptive choreographies, runtime conditions serve a different purpose: they trigger rollback or replacement [18,19,23].

We use MSCs as our causal model [3, 31]. Logics over MSCs and related traces include [1, 11, 13, 37]. Past-CTL gives the knowable causal past an event-structure semantics and translates it to distributed field-calculus monitors [4]. Its modalities quantify over causal paths. A survey of distributed runtime verification [24] covers decentralized LTL and knowledge-based monitoring [6, 26, 40]. Vector clocks also appear in message-sequence enforcement and causal monitoring of multithreaded programs [43, 44, 48].

8 Conclusion

We integrated PT-DTL-style past-time operators over visible causal history into workflow guards for choices owned by one lifeline. We proved the monitor correct for the MSC semantics and mechanized the proof in Lean. Future work will study more workflows and guard generation from natural-language specifications.

Use of AI assistance. AI coding agents (OpenAI Codex and Anthropic Claude) assisted with the Lean formalization, experiments, and manuscript editing. The author developed and takes full responsibility for the work. Lean kernel-checks the formalization without `sorry` or custom axioms. The experiment artifacts are public.

Acknowledgments. This work was partly supported by "France 2030", managed by ANR (ANR-23-PEIA-0006). We thank the anonymous reviewers for comments that substantially improved the paper.

References

1. Adsul, B., Gastin, P., Kulkarni, S., Weil, P.: An expressively complete local past propositional dynamic logic over Mazurkiewicz traces and its applications. In: Sobocinski, P., Dal Lago, U., Esparza, J. (eds.) Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2024, Tallinn, Estonia, July 8-11, 2024. pp. 2:1–2:13. ACM (2024)
2. Alamdari, P.A., Klassen, T.Q., McIlraith, S.A.: Formal methods meet LLMs: Auditing, monitoring, and intervention for compliance of advanced AI systems. In: Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency (FAccT 2026). pp. 4162–4198. ACM, Montreal, QC, Canada (Jun 2026)
3. Alur, R., Yannakakis, M.: Model checking of message sequence charts. In: Baeten, J.C.M., Mauw, S. (eds.) CONCUR '99: Concurrency Theory, 10th International Conference, Eindhoven, The Netherlands, August 24-27, 1999, Proceedings. pp. 114–129. Lecture Notes in Computer Science, Springer (1999)
4. Audrito, G., Damiani, F., Stolz, V., Torta, G., Viroli, M.: Distributed runtime verification by past-CTL and the field calculus. J. Syst. Softw. **187**, 111251 (2022)
5. Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to runtime verification. In: Bartocci, E., Falcone, Y. (eds.) Lectures on Runtime Verification - Introductory and Advanced Topics, pp. 1–33. Lecture Notes in Computer Science, Springer (2018)
6. Bauer, A., Falcone, Y.: Decentralised LTL monitoring. Formal Methods Syst. Des. **48**(1-2), 46–93 (2016)
7. Bauer, A., Leucker, M., Schallhart, C.: Runtime verification for LTL and TLTL. ACM Trans. Softw. Eng. Methodol. **20**(4), 14:1–14:64 (2011)
8. Bocchi, L., Chen, T., Demangeon, R., Honda, K., Yoshida, N.: Monitoring networks through multiparty session types. Theor. Comput. Sci. **669**, 33–58 (2017)
9. Bocchi, L., Honda, K., Tuosto, E., Yoshida, N.: A theory of design-by-contract for distributed multiparty interactions. In: Gastin, P., Laroussinie, F. (eds.) CONCUR 2010 - Concurrency Theory, 21th International Conference, CONCUR 2010, Paris, France, August 31-September 3, 2010. Proceedings. pp. 162–176. Lecture Notes in Computer Science, Springer (2010)
10. Bollig, B., Fortin, M., Gastin, P.: Gossiping in message-passing systems. CoRR **abs/1802.08641** (2018)
11. Bollig, B., Fortin, M., Gastin, P.: Communicating finite-state machines, first-order logic, and star-free propositional dynamic logic. J. Comput. Syst. Sci. **115**, 22–53 (2021)
12. Bollig, B., Függer, M., Nowak, T.: Provable coordination for LLM agents via message sequence charts. CoRR **abs/2604.17612** (2026), to appear in ISoLA 2026
13. Bollig, B., Kuske, D., Meinecke, I.: Propositional dynamic logic for message-passing systems. Log. Methods Comput. Sci. **6**(3) (2010)
14. Burlò, C.B., Francalanza, A., Scalas, A.: On the monitorability of session types, in theory and practice. In: 35th European Conference on Object-Oriented Programming (ECOOP 2021). Leibniz International Proceedings in Informatics, vol. 194, pp. 20:1–20:30. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2021)
15. Carbone, M., Montesi, F.: Deadlock-freedom-by-design: multiparty asynchronous global programming. In: Giacobazzi, R., Cousot, R. (eds.) The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013. pp. 263–274. ACM (2013)

16. Chen, F., Rosu, G.: Java-MOP: A monitoring oriented programming environment for Java. In: Halbwachs, N., Zuck, L.D. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 11th International Conference, TACAS 2005, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2005, Edinburgh, UK, April 4-8, 2005, Proceedings. pp. 546–550. Lecture Notes in Computer Science, Springer (2005)
17. Chen, T., Bocchi, L., Deniélou, P., Honda, K., Yoshida, N.: Asynchronous distributed monitoring for multiparty session enforcement. In: Bruni, R., Sassone, V. (eds.) Trustworthy Global Computing - 6th International Symposium, TGC 2011, Aachen, Germany, June 9-10, 2011. Revised Selected Papers. pp. 25–45. Lecture Notes in Computer Science, Springer (2011)
18. Coppo, M., Dezani-Ciancaglini, M., Venneri, B.: Self-adaptive multiparty sessions. Service Oriented Computing and Applications **9**(3–4), 249–268 (2015)
19. Dalla Preda, M., Gabbrielli, M., Giallorenzo, S., Lanese, I., Mauro, J.: Dynamic choreographies: Theory and implementation. Logical Methods in Computer Science **13**(2) (2017)
20. Dawes, J.H., Reger, G.: Specification of temporal properties of functions for runtime verification. In: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing (SAC 2019). pp. 2206–2214. ACM (2019)
21. Demangeon, R., Honda, K., Hu, R., Neykova, R., Yoshida, N.: Practical interruptible conversations: distributed dynamic verification with multiparty session types and Python. Formal Methods Syst. Des. **46**(3), 197–225 (2015)
22. Fidge, C.J.: Timestamps in message-passing systems that preserve the partial ordering. In: Proceedings of the 11th Australian Computer Science Conference. Australian Computer Science Communications, vol. 10, pp. 56–66 (Feb 1988)
23. Francalanza, A., Mezzina, C.A., Tuosto, E.: Towards choreographic-based monitoring. In: Ulidowski, I., Lanese, I., Schultz, U.P., Ferreira, C. (eds.) Reversible Computation: Extending Horizons of Computing, Lecture Notes in Computer Science, vol. 12070, pp. 128–150. Springer (2020)
24. Francalanza, A., Pérez, J.A., Sánchez, C.: Runtime verification for decentralised and distributed systems. In: Bartocci, E., Falcone, Y. (eds.) Lectures on Runtime Verification - Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457, pp. 176–210. Springer (2018)
25. Gopinathan, K., Feser, J., Naim, M., Tavares, Z., Bingham, E.: Pact: A choreographic language for agentic ecosystems. In: Proceedings of the 2nd International Workshop on Choreographic Programming (CP 2026) (2026), to appear. Also available as arXiv:2605.03143
26. Graf, S., Peled, D.A., Quinton, S.: Monitoring distributed systems using knowledge. In: Bruni, R., Dingel, J. (eds.) Formal Techniques for Distributed Systems - Joint 13th IFIP WG 6.1 International Conference, FMOODS 2011, and 31st IFIP WG 6.1 International Conference, FORTE 2011, Reykjavik, Iceland, June 6-9, 2011. Proceedings. pp. 183–197. Lecture Notes in Computer Science, Springer (2011)
27. Havelund, K., Roşu, G.: Synthesizing monitors for safety properties. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2002). Lecture Notes in Computer Science, vol. 2280, pp. 342–356. Springer (2002)
28. van den Heuvel, B., Pérez, J.A., Dobre, R.A.: Monitoring blackbox implementations of multiparty session protocols. In: Katsaros, P., Nenzi, L. (eds.) Runtime Verification - 23rd International Conference, RV 2023, Thessaloniki, Greece, October 3-6, 2023, Proceedings. pp. 66–85. Lecture Notes in Computer Science, Springer (2023)

29. Hirsch, A.K., Garg, D.: Pirouette: higher-order typed functional choreographies. Proc. ACM Program. Lang. **6**(POPL), 1–27 (2022)
30. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. J. ACM **63**(1), 9:1–9:67 (2016)
31. ITU-T: Recommendation Z.120: Message sequence chart (MSC). Tech. rep., International Telecommunication Union (2011)
32. Kamath, A., Zhang, S., Xu, C., Ugare, S., Singh, G., Misailovic, S.: Enforcing temporal constraints for LLM agents. CoRR **abs/2512.23738** (2025)
33. Lamport, L.: Time, clocks, and the ordering of events in a distributed system. Commun. ACM **21**(7), 558–565 (1978)
34. Leucker, M., Schallhart, C.: A brief account of runtime verification. J. Log. Algebraic Methods Program. **78**(5), 293–303 (2009)
35. Liu, Y.A., Stoller, S.D., Lin, B.: From clarity to efficiency for distributed algorithms. ACM Transactions on Programming Languages and Systems **39**(3), 12:1–12:41 (2017)
36. Mattern, F.: Virtual time and global states of distributed systems. In: Parallel and Distributed Algorithms: Proceedings of the International Workshop on Parallel and Distributed Algorithms. pp. 215–226. North-Holland, Amsterdam (1989)
37. Mazurkiewicz, A.W.: Trace theory. In: Brauer, W., Reisig, W., Rozenberg, G. (eds.) Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II, Proceedings of an Advanced Course, Bad Honnef, Germany, 8-19 September 1986. pp. 279–324. Lecture Notes in Computer Science, Springer (1986)
38. Meenakshi, B., Ramanujam, R.: Reasoning about message passing in finite state environments. In: Montanari, U., Rolim, J.D.P., Welzl, E. (eds.) Automata, Languages and Programming, 27th International Colloquium, ICALP 2000, Geneva, Switzerland, July 9-15, 2000, Proceedings. pp. 487–498. Lecture Notes in Computer Science, Springer (2000)
39. Montesi, F.: Introduction to Choreographies. Cambridge University Press (2023)
40. Mostafa, M., Bonakdarpour, B.: Decentralized runtime verification of LTL specifications in distributed systems. In: 2015 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2015, Hyderabad, India, May 25-29, 2015. pp. 494–503. IEEE Computer Society (2015)
41. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Platzer, A., Sutcliffe, G. (eds.) Automated Deduction - CADE 28 - 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings. pp. 625–635. Lecture Notes in Computer Science, Springer (2021)
42. Paduraru, C., Bouruc, P.L., Stefanescu, A.: A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance. In: Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2026). pp. 420–427. SciTePress (2026)
43. Ponce-de-León, H., Haar, S., Longuet, D.: Distributed testing of concurrent systems: Vector clocks to the rescue. In: Ciobanu, G., Méry, D. (eds.) Theoretical Aspects of Computing - ICTAC 2014 - 11th International Colloquium, Bucharest, Romania, September 17-19, 2014. Proceedings. pp. 369–387. Lecture Notes in Computer Science, Springer (2014)
44. Samadi, M., Ghassemi, F., Khosravi, R.: Decentralized runtime verification of message sequences in message-based systems. Acta Informatica **60**(2), 145–178 (2023)
45. Scheffel, T., Schmitz, M.: Three-valued asynchronous distributed runtime verification. In: 12th ACM/IEEE International Conference on Formal Methods and Models for Codesign (MEMOCODE 2014). pp. 52–61. IEEE (2014)

46. Schneider, F.B.: Enforceable security policies. ACM Trans. Inf. Syst. Secur. **3**(1), 30–50 (2000)
47. Sen, K., Vardhan, A., Agha, G., Roşu, G.: Efficient decentralized monitoring of safety in distributed systems. In: Proceedings of the 26th International Conference on Software Engineering (ICSE 2004). pp. 418–427. IEEE Computer Society (2004)
48. Sen, K., Vardhan, A., Agha, G., Rosu, G.: Decentralized runtime analysis of multithreaded applications. In: 20th International Parallel and Distributed Processing Symposium (IPDPS 2006), Proceedings, 25-29 April 2006, Rhodes Island, Greece. IEEE (2006)
49. Shen, G., Kashiwa, S., Kuper, L.: HasChor: Functional choreographic programming for all (functional pearl). Proc. ACM Program. Lang. **7**(ICFP), 541–565 (2023)
50. Singh, M.P.: Distributed enactment of multiagent workflows: Temporal logic for web service composition. In: The Second International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS 2003. pp. 907–914. ACM (2003)
51. Stolz, V., Bodden, E.: Temporal assertions using AspectJ. Electronic Notes in Theoretical Computer Science **144**(4), 109–124 (2006)
52. Wang, H., Poskitt, C.M., Sun, J.: AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026). pp. 1–12. ACM (2026)
53. Zhang, Y., Emma, S.Y., En, A.L.J., Dong, J.S.: RvLLM: LLM runtime verification with domain knowledge. In: Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025 (2025)